

A Domain-Specific Language and Web Interface for Geometric Simplicial Complexes in PolyLogicA

Simone Riccio

July 12, 2026

Abstract

This report presents the design and implementation of a novel Domain-Specific Language (DSL) and web interface tailored for the construction and manipulation of geometric simplicial complexes. Developed to interface directly with the PolyLogicA branch of the VoxLogicA project, this tool allows users to define topological spaces in Euclidean coordinates, apply geometric transformations, and evaluate spatial logic queries. We detail the language grammar, the abstract syntax tree evaluation, and the topological serialization process required to generate the Hasse diagram (poset) representations necessary for spatial model checking.

1 Introduction

Spatial logic provides a robust mathematical framework for reasoning about the properties of space, topological relations, and geometric configurations [1]. Within the context of the VoxLogicA project—and specifically its PolyLogicA branch—evaluating these spatial properties requires translating geometric constructs into formal topological structures, typically represented as partially ordered sets (posets).

Constructing these posets manually is highly error-prone and unintuitive. To bridge this gap, this project introduces a custom Domain-Specific Language (DSL) that allows users to express complex geometric structures using simple Euclidean points and high-level operations (e.g., union, star, boundary). We implemented a Python-based interpreter that parses this DSL, evaluates the geometric expressions, computes the strict downward topological closures, and automatically serializes the data into PolyLogicA’s expected JSON format. Furthermore, we provide a web interface leveraging Uvicorn and Three.js to offer real-time visual feedback and model checker results.

2 Preliminaries, Notation and Definitions

In this section, we introduce the mathematical definitions that form the core of our evaluation engine.

Definition 1 (Geometric Simplex). *A geometric k -simplex is the convex hull of $k + 1$ affinely independent points in Euclidean space. We denote a simplex by the set of its vertices $S = \{p_0, p_1, \dots, p_k\}$. The dimension of the simplex is k .*

Definition 2 (Simplicial Complex). *A geometric simplicial complex K is a set of simplices satisfying two conditions:*

1. *Every face of a simplex in K is also in K .*
2. *The non-empty intersection of any two simplices in K is a face of both.*

In our implementation, a `GeometricComplex` is strictly represented by its maximal simplices. To ensure it satisfies the first condition (closure under faces), our serialization algorithm generates the downward closure by computing all combinatorial subsets of the vertices.

Remark 1 (Topological Operators). *While standard operations like translation, rotation, and scaling preserve the structure of the complex, operations like **boundary** and **star** require special interpretation in our geometric context. In standard algebraic topology, the boundary operator maps to a chain complex. In our DSL, we define the geometric boundary of K as the complex containing all simplices strictly smaller than the maximum dimension of K . Similarly, the **star** of a subcomplex $L \subseteq K$ is defined as the set of all simplices in K that contain any simplex of L .*

3 The Domain-Specific Language (DSL)

To allow users to intuitively define these spaces, we engineered a declarative grammar using the Lark parsing library. The language supports variable assignments, point declarations, complex definitions, and functional operator calls.

Listing 1: Core Grammar Rules of the DSL

```

program: statement*

?statement: point_decl | complex_decl | assign | render_stmt

point_decl: "point" IDENT "=" tuple_coord
complex_decl: "complex" IDENT "=" expr
render_stmt: "render" IDENT

?expr: op_call | IDENT | vertices_list | tuple_coord | NUMBER | "("
      expr ")"
op_call: OP "(" arg_list? ")"

```

The execution model relies on an *Environment* mapping identifiers to internal data structures (Points, GeometricComplexes, and Operators). When an expression like `complex t1 = [p0, p1, p2]` is evaluated, the interpreter validates the points and instantiates a `GeometricComplex` defined by the topological downward closure of the provided vertices.

References

- [1] Marco Aiello, Ian Pratt-Hartmann, and Johan van Benthem, *Handbook of Spatial Logic*, Springer, 2007.
- [2] Vincenzo Ciancia et al., *VoxLogicA: A Spatial Model Checker*,